Project: Customer Support Ticket Classifier

Objective: Build an end-to-end system that automatically classifies support tickets into issue types (billing, technical, account, etc.) and tags ticket sentiment (positive/negative). Include full MLOps: experiment tracking (MLflow), pipeline orchestration (Airflow), and production deployment (Docker + Kubernetes).

---

10 Clear Steps

Step 1 — Problem definition & scope

- Tasks:
    - Multi-class classification of tickets into predefined issue types (e.g., Billing, Technical, Account, Shipping, Product Info, Others).
    - Binary sentiment tagging (Positive / Negative) per ticket.
- Scope: data ingestion → preprocessing → model development (SVM, ANN, CNN) → evaluation → Airflow orchestration → MLflow tracking → Docker/K8s deployment → monitoring.

Step 2 — Dataset acquisition & EDA

- Use the **Customer Support Tickets Dataset** (or similar: internal logs / HuggingFace dataset).
- Perform Exploratory Data Analysis:
    - Class balance, ticket length distribution, common tokens/phrases per class.
    - Missing values, noisy text (html, emojis), and meta fields (timestamp, category if available).
- Decide taxonomy of classes (fix label set) and percent split for train/val/test.

Step 3 — Preprocessing & label preparation

- Text cleaning: lowercase, strip HTML, remove special chars, normalize whitespace.
- Tokenization, stopword removal, lemmatization (or stemming if preferred).
- Normalize/collapse rare categories into “Other” if very sparse.

- Generate or confirm sentiment labels:
    - Use existing sentiment labels if available, otherwise bootstrap using rule-based or lexicon methods (VADER/TextBlob) and later refine with manual checks.
- Save cleaned dataset artifacts (CSV/Parquet) and vocab.

Step 4 — Feature engineering

- Classic features:
    - TF-IDF (unigrams + bigrams), document length, punctuation counts.
    - Optional: POS tag counts, presence of keywords, metadata (time of day, channel).
- Embedding features (for DL):
    - Pretrained Word2Vec/GloVe embeddings or train embeddings from scratch on the corpus.
    - Build padded sequences for ANN/CNN input.

Step 5 — Baseline model(s) with TF-IDF

- Train baseline(s) using TF-IDF:
    - **SVM (LinearSVC)** and/or **Logistic Regression**.
- Model selection:
    - Use cross-validation, grid search on hyperparameters (C, penalty).
    - Evaluate with Accuracy, Macro F1, per-class Precision/Recall.
- Log every run to **MLflow** (params, metrics, artifacts).

Step 6 — Deep learning models: ANN & CNN

- **ANN**:
    - Dense network on averaged embeddings or TF-IDF → Dense → Dropout → Softmax.
- **CNN**:

    - Embedding layer → multiple 1D conv filters (sizes 3/4/5) → max-pool → concat → dense → softmax.
- Training details:
    - Use early stopping, learning-rate schedule, class weighting/oversampling if classes imbalanced.
- Track training/validation curves and model artifacts in **MLflow**.

Step 7 — Sentiment analysis module

- Option A: Single multi-output model (predicts class + sentiment).
- Option B: Two separate models: one classifier for issue type, one binary sentiment classifier.
- Train sentiment classifier with TF-IDF + SVM baseline and upgrade to ANN/CNN if needed.
- Evaluate with Accuracy, Precision/Recall/F1 and track in MLflow.

Step 8 — Evaluation, error analysis & calibration

- Final metrics: Accuracy, Macro F1 (important for imbalance), per-class precision/recall, confusion matrix.
- Additional checks:
    - Confidence calibration (predictive probabilities).
    - Error analysis: sample misclassified tickets, analyze causes (ambiguous text, missing context).
    - A/B test candidate models on holdout set.
- Decide final model(s) to register in MLflow Model Registry.

Step 9 — Orchestration & CI for training (Airflow)

- Build Airflow DAG(s) for full pipeline:
    1. Ingest new tickets (or batch upload).
    2. Preprocess & store cleaned data.

3. Train baseline & DL models (or conditional: only training when new labeled data exists).
4. Evaluate models and register best model in MLflow.
5. Deploy model (update inference service) or save model artifact for manual promotion.

- Schedule: hourly/ daily ingest, weekly retrain (or on-demand via Airflow UI).

Step 10 — Containerization, deployment & monitoring

- **Dockerize** services:
  - Preprocessing service (batch jobs).
  - Training service (optional).
  - Inference API (FastAPI/Flask) that accepts ticket text → returns { category, sentiment, confidence }.
- **Kubernetes** deployment:
  - Deploy inference as a scalable Deployment + Service.
  - Use Horizontal Pod Autoscaler if load varies.
  - Add logging (ELK/Cloud logging) and basic monitoring (Prometheus + Grafana) for request rates, latencies, error rates.
- Set up structured logging for predictions so drift/feedback can be analyzed and fed back for retraining.

Deliverables (what students submit)

1. **Project Report** (PDF) — problem, dataset, preprocessing, models, experiments, chosen model, results, conclusions, limitations, future work.
2. **Codebase (GitHub repo)** with clear structure:
   - data/ (raw + processed samples)
   - notebooks/ (EDA, model prototyping)
   - preprocessing/ (scripts)
   - models/ (SVM, ANN, CNN training code)
   - training/ (training pipelines)

- inference/ (API code)
- airflow_dags/
- docker/ (Dockerfiles)
- k8s/ (Kubernetes manifests)

3. **MLflow tracking artifacts** (runs, best model registered).
4. **Airflow DAG file(s)** and screenshots of DAG runs.
5. **Docker images** (or Dockerfiles) and **Kubernetes manifests** (Deployment, Service, optional HPA).
6. **REST API** (endpoint docs) + Demo script (curl/postman) to call /predict.
7. **Evaluation artifacts**: confusion matrices, per-class metrics, training logs, ROC/PR curves for sentiment model.
8. **Test suite**: a small set of unit/integration tests for preprocessing and inference endpoints.
9. **Deployment runbook**: instructions to build images, apply k8s manifests, and run the demo.
10. **Presentation slides & demo video** (5–10 min) showing the pipeline, results, and a live demo of classifying sample tickets.